Data Structure

- The way data are organized in a computers memory and logical or mathematical model of a particular organization of data is called Data Structure.

- There are two classifications of data structures, depending on their usage.

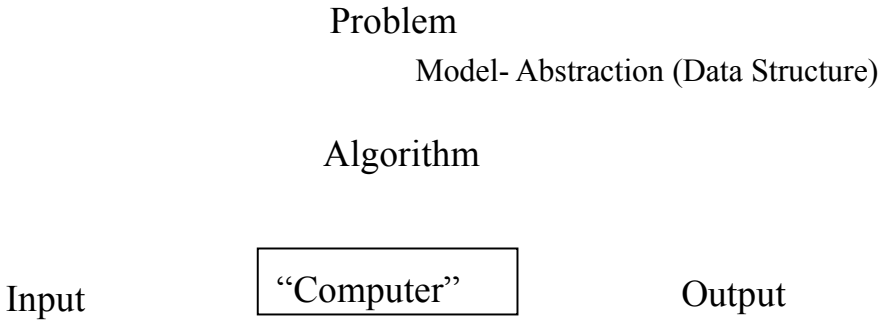
  - Linear Data Structures: if its elements form a sequence or linear list. Ex. Arrays, Stacks, Queues and linked lists.
  - Non-linear Data Structures: , it is elements do not form a sequence. Ex. Trees and Graphs.

- Data Structure have Traversing, Deleting, Inserting, Merging, and Sorting operation

Cont'...

Algorithm is a sequence of unambiguous instructions for solving a problem, i.e., for obtaining a required output for any legitimate input in a finite amount of time.

- The sequence of computational steps to solve a problem is an algorithm.
- Therefore, a program is nothing but data structures plus algorithms.
-



- Properties of an algorithm

- Finiteness: Algorithm must complete after a finite number of steps.
- Definiteness: Each step must be clearly defined, having one and only one interpretation. At each point in computation, one should be able to tell exactly what happens next.
- Sequence: Each step must have a unique defined preceding and succeeding step. The first step (start step) and last step (halt step) must be clearly noted.
- Feasibility: It must be possible to perform each instruction.
- Correctness: It must compute correct answer for all possible legal inputs.
- Language Independence: It must not depend on any one programming language.
- Completeness: It must solve the problem completely.
- Effectiveness: It must be possible to perform each step exactly and in a finite amount of time. Steps are sufficiently simple and basic

Abstract Data Types

- What is the d/f b/w the data item and data structure?
- A type is a collection of values. Example Boolean
- A data item is a piece of information or a record whose value is drawn from a type.
- A data item is said to be a member of a type.
- ADT is the realization of a data type as a software component.
- The interface of the ADT is defined in terms of a type and a set of operations on that type.
- The behavior of each operation is determined by its inputs and outputs.

Cont'...

- An ADT does not specify how the data type is implemented.
- The implementation details are hidden from the user of the ADT
- Data structure often refers to data stored in a computer's main memory.
- It is the implementation for an ADT.
- File structure often refers to the organization of data on peripheral storage, such as a disk drive or CD-ROM.

- ADT

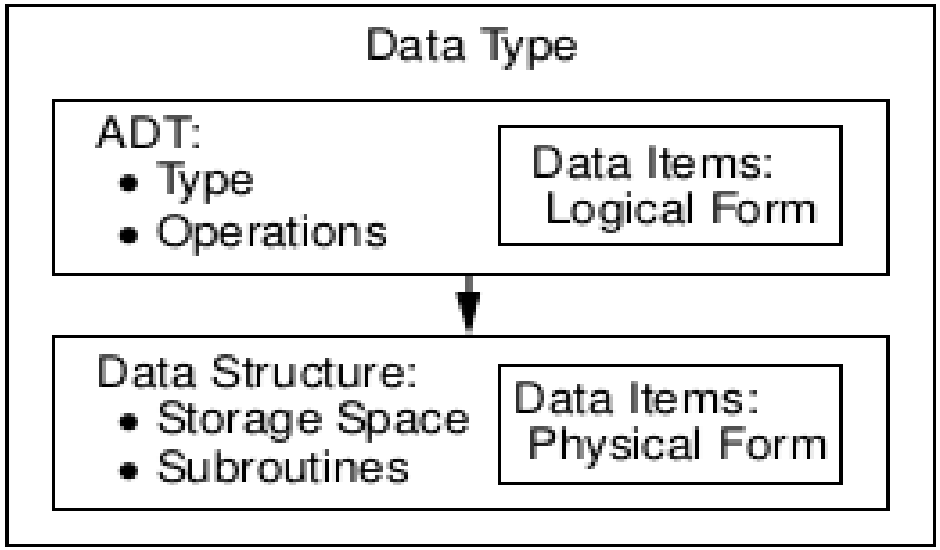


Figure 1.1 The relationship between data items, abstract data types, and data structures. The ADT defines the logical form of the data type. The data structure implements the physical form of the data type.

Algorithm Analysis Concepts

Algorithm analysis refers to the process of determining how much computing time and storage that algorithms will require.
In other words, it's a process of predicting the resource requirement of algorithms in a given environment.
The main resources are:

- Computational Time (Compile Time and Running Time)
- Memory Usage
- Communication Bandwidth

Algorithm complexity can be further divided into two types: time complexity and space complexity.

- Time complexity, refers to the time taken by the algorithm to complete its execution.
- Space complexity refers to the memory occupied by the algorithm.

Space Complexity: An algorithm is the amount of memory it needs to run to completion.

- Fixed part: independent of the characteristics of the input and output. space for the code, space for simple variables, space for the fixed size component variables and constants...
- Variable part: It consists of the space needed by component variables whose size is dependent on the particular problem instance being solved, the space needed by referenced variables and the recursion stack space.

Time Complexity: an algorithm is the amount of computer time it needs to run to completion.

- It will be difficult for us to determine the run time of a program exactly.
- Because the run time may depend on
  - Specific Processor speed
  - Type of compiler used
  - Current Processor Load
  - Specific data for a particular run of the program (Logical size of the program)
    - ➔Input Size
    - ➔Input Properties
  - Available memory.
  - Operating Environment

Asymptotic Analysis

- It deal with how the running time of an algorithm increases with the size of the input in the limit, as the size of the input increases without bound.
- Generally, the complexity is investigated in three cases. They are:
  - *Worst Case:* The maximum value of the function f(n) for any possible input.
  - *Average Case:* The expected value of f(n)
  - *Best Case:* Minimum possible value of f(n)
- These are several kings of mathematical notations. These are:
  - ✓ Big-Oh Notation (O)
  - ✓ Big-Omega Notation (Ω)
  - ✓ Theta Notation (Θ)
  - ✓ Little-o Notation (o)
  - ✓ Little-Omega Notation (ω)

Big-O Notation

- Big-Oh notation is a way of comparing algorithms.
- Used for computing the complexity of algorithms.
- The amount of time that it takes for computer program to run .
- It's only concerned with what happens for very a large value of n.
- W*ays to describe the upper bound for an algorithm and the lower bound for an algorithm.*
- *Upper bound find an equation for the maximum cost of a particular class of inputs of size n.*

Comparison of Algorithms

- Comprision Algorthim

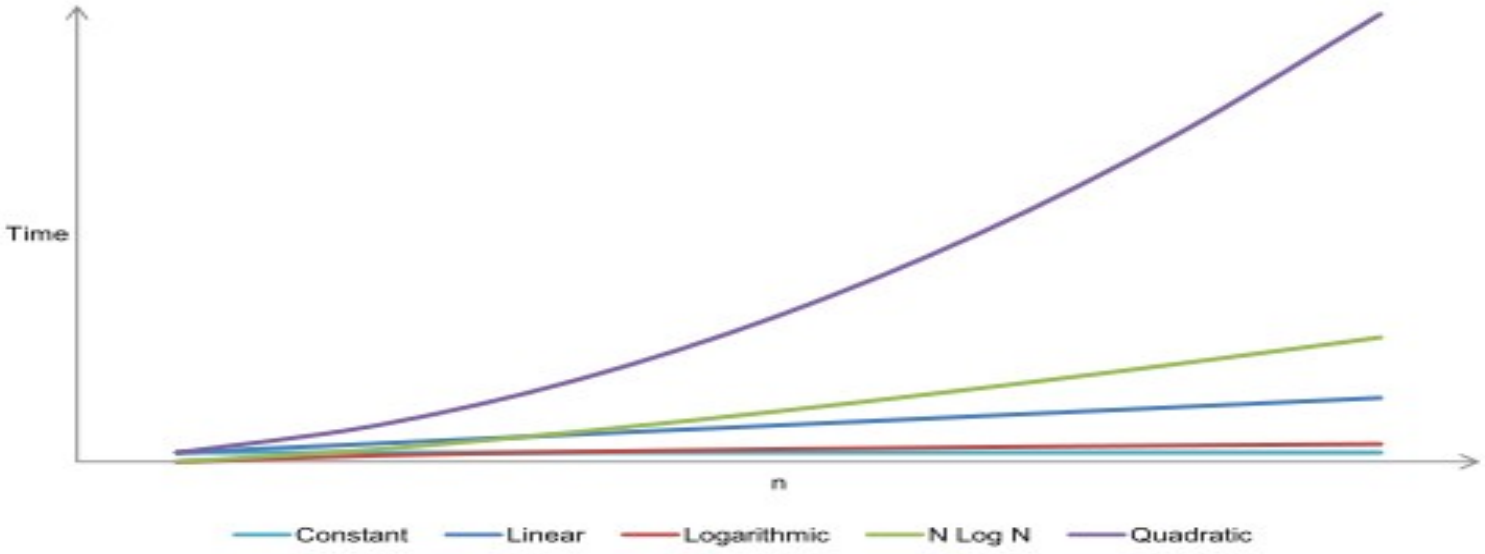


Figure 1.1: Algorithmic run time expansion

The following list explains some of the most common big Oh notations:

- O(1) constant: the operation doesn't depend on the size of its input,
- e.g. adding a node to the tail of a linked list where we always maintain a pointer to the tail node.
- O(n) linear: the run time complexity is proportionate to the size of n.
- O(log n) logarithmic: normally associated with algorithms that break the problem into smaller chunks per each invocation, e.g. searching a binary search tree.
- O(nlogn): usually associated with an algorithm that breaks the problem into smaller chunks per each invocation, and then takes the results of these smaller chunks and stitches them back together, e.g. quick sort.
- O(n2) quadratic: e.g. bubble sort.
- $O(n^3)$ cubic: very rare.
- $O(2^n)$ exponential: incredibly rare.

✓ Big-Omega Notation (Ω)

- A lower bound on the computing time of an algorithm
- Used to represent best case running time of algorithm
- A function f(n) is ␣(g (n)) if there exist constants c and k ϵ $\mathscr{R}$+ such that f(n) >=c. g(n) for all n>=k.
- f(n)= Ω(g(n)) means that f(n) is greater than or equal to some constant multiple of g(n) for all values of n greater than or equal to some k.

Examples: 1. If f(n)=3n + n = Ω(n) as 3n+3 ≥ for n ≥ 1.

2. If f(n) =$n^{2,}$ then f(n)= Ω(n)

Θ Notation

- In some cases, the worst case and best case time complexities are equal i.e., f(n)=W(g(n)) and f(n)=O(g(n)).
- If f(n) = θ (g(n)) then g(n) is both upper and lower bound on f(n) (g(n) is an asymptotically tight bound for f(n)).
- The best and worst cases require the same amount of time within a constant factor.

In simple terms, f(n)= ⊣ (g(n)) means that f(n) and g(n) have the same rate of growth.

Analysis Rules

1) We assume an arbitrary time unit.

2) Execution of one of the following operations takes time 1:

- Assignment Operation
- Single Input/Output Operation
- Single Boolean Operations
- Single Arithmetic Operations
- Function Return

1) 3) Running time of a selection statement (if, switch) is the time for the condition evaluation + the maximum of the running times for the individual clauses in the selection.

2) 4) Loops: Running time for a loop is equal to the running time for the statements inside the loop * number of iterations.

3) The total running time of a statement inside a group of nested loops is the running time of the statements multiplied by the product of the sizes of all the loops.

4) For nested loops, analyze inside out.

5) Always assume that the loop executes the maximum number of iterations possible.

6) 5) Running time of a function call is 1 for setup + the time for any parameter calculations + the time required for the execution of the function body.